PYTHON PROGRAMMING

Activity-Based Learning

10 Case Studies • Algorithms • Python Solutions

STUDENT DETAILS

Name: J. Charanya
Roll No: 26000087 | Section: CSE-2

Prepared using the assigned Python syllabus

Selected Case Studies

Ten practical problems chosen from the supplied Python Activity-Based Learning sheet.

01. Scholarship Eligibility

Conditions • logical operators • lists

02. Progressive Electricity Billing

Arithmetic • slabs • loops

03. Hospital Billing

Variables • conditions • totals

04. Employee Payroll

Arithmetic • formatted output

05. Mobile Plan Optimization

Lists • comparisons • minimum cost

06. Railway Fare Calculator

Conditions • age concessions • loops

07. Retail Billing

Lists • discounts • taxes

08. Manufacturing Profit

Cost analysis • target checking

09. Student Marks Matrix

Nested lists • averages • ranking

10. Tuple Product Catalogue

Tuples • validation • unpacking

Project Overview

The solutions below are based on the supplied case-study assignment and assigned syllabus.

Purpose

Apply beginner Python concepts to realistic college, household, business, and student-data problems.

Core concepts

Variables, data types, operators, conditions, loops, strings, lists, nested lists, tuples, and unpacking.

Method

For each case: problem, objectives, assumptions, algorithm, flowchart logic, pseudocode, code, test, conclusion.

Selected case studies from the supplied PDF

- Case studies are selected from the assignment list (including scholarship, billing, payroll, fare, retail, manufacturing, marks matrix, and tuple catalogue).
- Rates, thresholds, and policies are stated as assumptions for demonstration because the assignment sheet does not supply numeric policies.

1. University Scholarship Allocation

Case study overview and computational thinking

Problem statement

Select students who satisfy aggregate marks, subject minimum, attendance, and family-income rules.

Objectives

- Check every eligibility condition
- Use a list to store applicant records
- Rank eligible applicants by aggregate marks

Assumptions

- Aggregate ≥ 80%
- Each subject ≥ 70%
- Attendance ≥ 75%
- Annual family income ≤ ₹2,50,000

Algorithm

1. Read applicant details
2. Check all four conditions
3. Store eligible records
4. Sort by aggregate, highest first

Computational thinking: break the problem into inputs, rules, calculations, and a clear output.

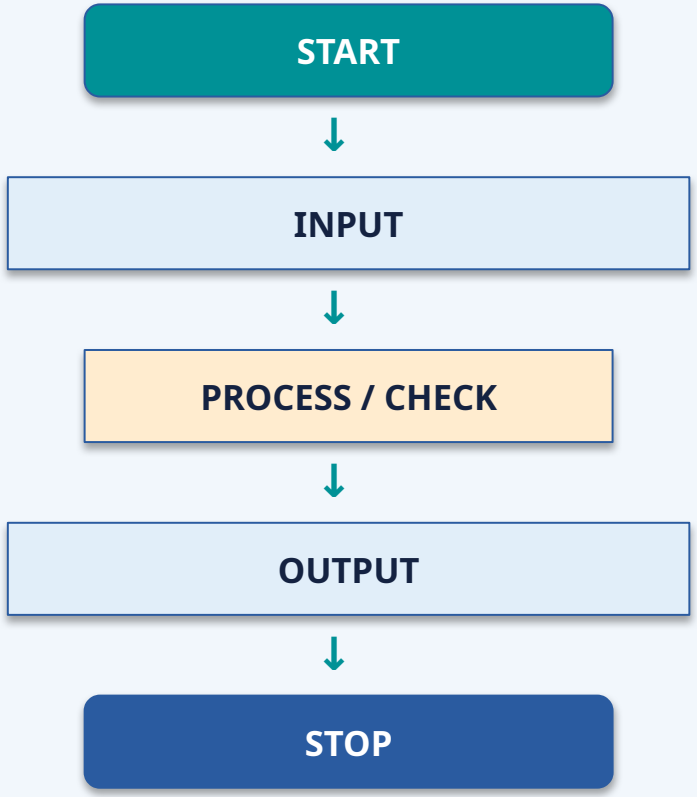
1. University Scholarship Allocation — Program Design

Pseudocode and flowchart logic

Pseudocode

START → Read applicant → Check marks, attendance, income → Eligible? → Add to list → Rank → STOP

Flowchart (logic)



Decision branches are implemented using if / elif / else.

1. University Scholarship Allocation — Python Solution

Complete beginner-friendly source code

```
                                        applicants = [
    ("Asha", 88, 76, 82, 90, 80, 78, 72, 82, 220000),
    ("Ravi", 75, 80, 77, 79, 81, 74, 78, 78, 180000),
    ("Meena", 92, 90, 88, 94, 91, 90, 89, 91, 200000)
]
eligible = []
for a in applicants:
    name, s1, s2, s3, s4, s5, s6, s7, agg, income = a
    attendance = 82 if name != "Ravi" else 88
    if agg >= 80 and min(s1,s2,s3,s4,s5,s6,s7) >= 70 and attendance >= 75 and income <= 250000:
        eligible.append((name, agg))
eligible.sort(key=lambda x: x[1], reverse=True)
print("Eligible scholarship applicants:")
for name, marks in eligible:
    print(name, marks)
```

1. University Scholarship Allocation — Testing & Conclusion

Example test case and expected result

Test case

Meena: 91%, Asha: 82%, Ravi: not eligible (aggregate below 80%).

Conclusion

Conditional statements and list sorting can shortlist and rank scholarship applicants consistently.

Validation note

Run the code in Python and compare actual output with the expected result; adjust inputs to test boundary cases.

2. Progressive Electricity Billing

Case study overview and computational thinking

Problem statement

Calculate a household electricity bill using consumption slabs, a fixed charge, and a rebate.

Objectives

- Apply progressive slab rates
- Use loops and arithmetic operators
- Calculate final payable amount

Assumptions

- First 100 units: ₹1.5/unit
- Next 100 units: ₹2.5/unit
- Above 200: ₹4/unit
- Fixed charge ₹50; rebate 5% if bill before fixed charge exceeds ₹500

Algorithm

1. Read units
2. Calculate charge for each slab
3. Add fixed charge
4. Apply eligible rebate

Computational thinking: break the problem into inputs, rules, calculations, and a clear output.

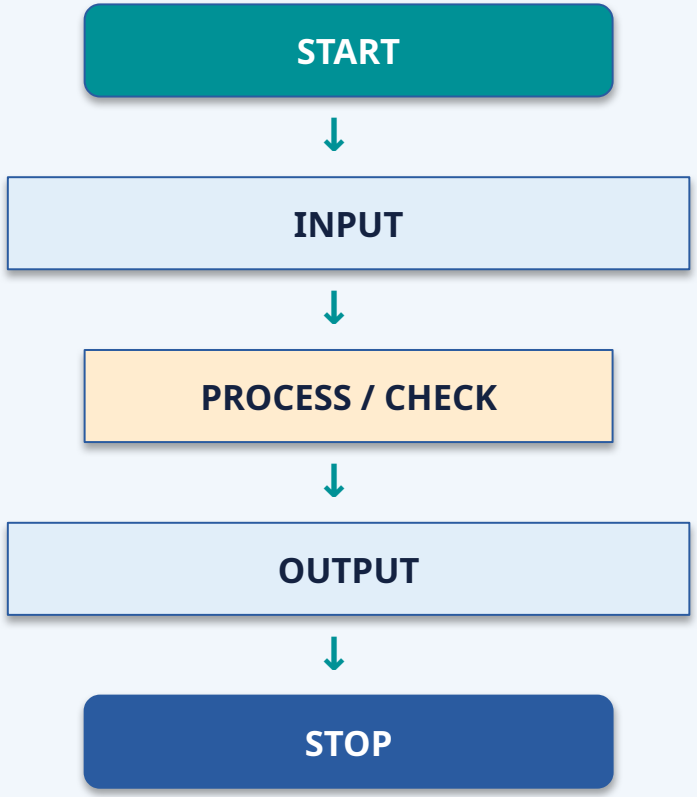
2. Progressive Electricity Billing — Program Design

Pseudocode and flowchart logic

Pseudocode

START → Read units → Calculate slab charges → Add fixed fee → Apply rebate if eligible → Print bill → STOP

Flowchart (logic)



Decision branches are implemented using if / elif / else.

2. Progressive Electricity Billing — Python Solution

Complete beginner-friendly source code

```
                                units = int(input("Enter units consumed: "))
if units < 0:
    print("Invalid units")
else:
    bill = 0
    if units <= 100:
        bill = units * 1.5
    elif units <= 200:
        bill = 100 * 1.5 + (units - 100) * 2.5
    else:
        bill = 100 * 1.5 + 100 * 2.5 + (units - 200) * 4
    amount = bill + 50
    if bill > 500:
        amount = amount * 0.95
    print("Energy charge: ₹", round(bill, 2))
    print("Final bill: ₹", round(amount, 2))
```

2. Progressive Electricity Billing — Testing & Conclusion

Example test case and expected result

Test case

Input: 250 units → energy charge ₹450.00; fixed charge ₹50; final bill ₹500.00.

Conclusion

Slab-based conditions calculate bills fairly and show how operators and selection statements work together.

Validation note

Run the code in Python and compare actual output with the expected result; adjust inputs to test boundary cases.

3. Hospital Billing and Patient Classification

Case study overview and computational thinking

Problem statement

Prepare a patient bill from consultation, diagnostic, and treatment charges, with a concession for eligible patients.

Objectives

- Add multiple service charges
- Validate non-negative inputs
- Apply a conditional concession

Assumptions

- Senior citizens aged 60+ receive 10% concession
- Concession applies to total service charges
- Negative charges are invalid

Algorithm

1. Read age and three charges
2. Validate values
3. Add charges
4. Apply concession when age ≥ 60
5. Print itemized total

Computational thinking: break the problem into inputs, rules, calculations, and a clear output.

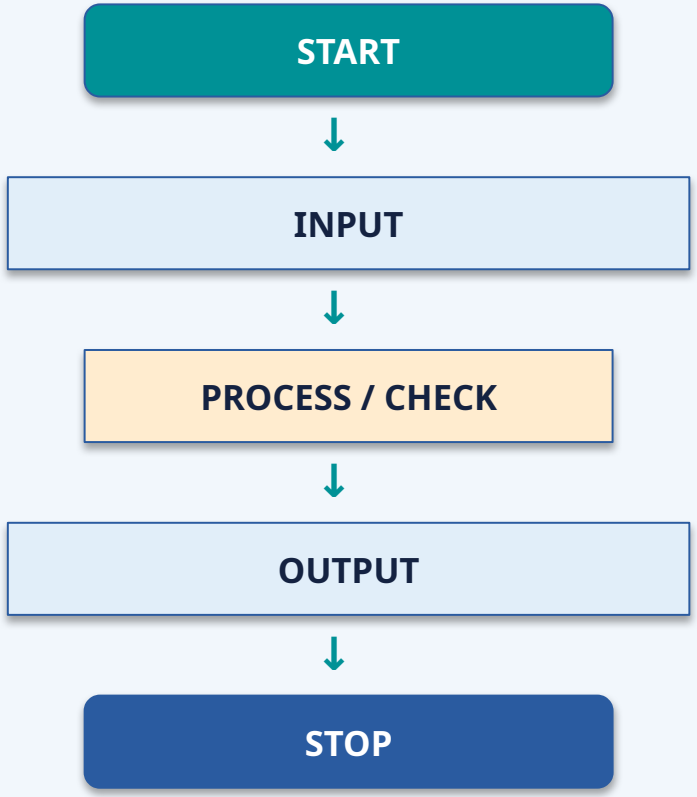
3. Hospital Billing and Patient Classification — Program Design

Pseudocode and flowchart logic

Pseudocode

START → Read age and charges → Validate → Total charges → Age ≥ 60? → Apply 10% → Display bill → STOP

Flowchart (logic)



Decision branches are implemented using if / elif / else.

3. Hospital Billing and Patient Classification — Python Solution

Complete beginner-friendly source code

```
                                age = int(input("Enter patient age: "))
consultation = float(input("Consultation charge: "))
diagnostic = float(input("Diagnostic charge: "))
treatment = float(input("Treatment charge: "))

if age < 0 or min(consultation, diagnostic, treatment) < 0:
    print("Invalid input")
else:
    total = consultation + diagnostic + treatment
    concession = total * 0.10 if age >= 60 else 0
    payable = total - concession
    print("Service total: ₹", round(total, 2))
    print("Concession: ₹", round(concession, 2))
    print("Amount payable: ₹", round(payable, 2))
```

3. Hospital Billing and Patient Classification — Testing & Conclusion

Example test case and expected result

Test case

Age 65; charges ₹300, ₹500, ₹1,200 → total ₹2,000; concession ₹200; payable ₹1,800.

Conclusion

Variables, validation, arithmetic, and if-else conditions support a clear hospital invoice.

Validation note

Run the code in Python and compare actual output with the expected result; adjust inputs to test boundary cases.

4. Employee Payroll and Deduction Analysis

Case study overview and computational thinking

Problem statement

Calculate gross salary from basic pay, allowances, overtime, and incentives, then deduct a percentage.

Objectives

- Compute gross salary
- Apply a deduction rule
- Display a salary statement

Assumptions

- HRA = 20% of basic
- Other allowance = 10% of basic
- Overtime and incentive are supplied amounts
- Deduction = 8% of gross salary

Algorithm

1. Read basic, overtime, incentive
2. Calculate allowances
3. Find gross pay
4. Calculate deduction and net pay

Computational thinking: break the problem into inputs, rules, calculations, and a clear output.

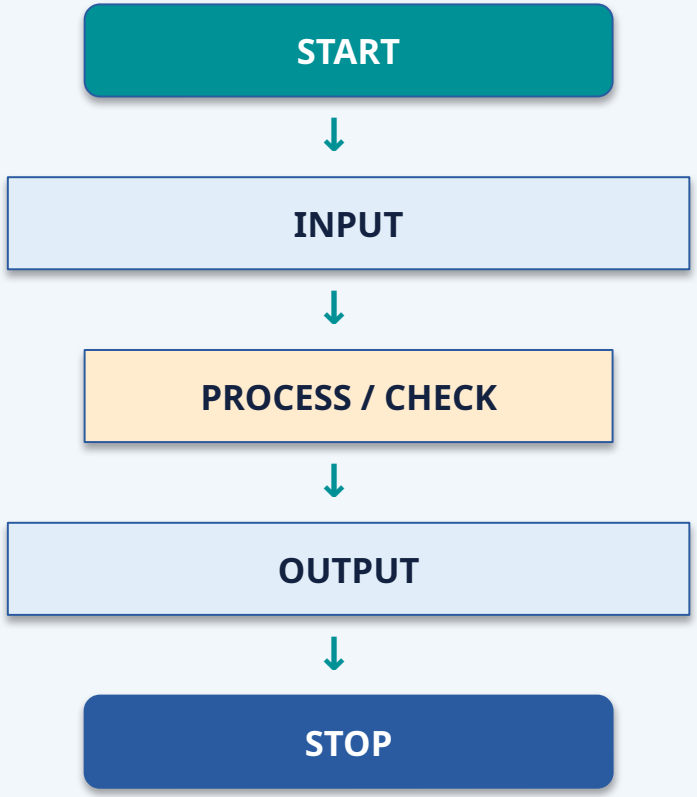
4. Employee Payroll and Deduction Analysis — Program Design

Pseudocode and flowchart logic

Pseudocode

START → Read salary inputs → Calculate HRA and allowance → Gross → Deduction → Net pay → STOP

Flowchart (logic)



Decision branches are implemented using if / elif / else.

4. Employee Payroll and Deduction Analysis — Python Solution

Complete beginner-friendly source code

```
                                basic = float(input("Basic salary: ₹"))
overtime = float(input("Overtime amount: ₹"))
incentive = float(input("Incentive: ₹"))

if min(basic, overtime, incentive) < 0:
    print("Invalid salary input")
else:
    hra = basic * 0.20
    allowance = basic * 0.10
    gross = basic + hra + allowance + overtime + incentive
    deduction = gross * 0.08
    net = gross - deduction
    print("HRA: ₹", round(hra, 2))
    print("Gross salary: ₹", round(gross, 2))
    print("Deduction: ₹", round(deduction, 2))
    print("Net salary: ₹", round(net, 2))
```

4. Employee Payroll and Deduction Analysis — Testing & Conclusion

Example test case and expected result

Test case

Basic ₹20,000; overtime ₹1,000; incentive ₹500 → gross ₹27,500; deduction ₹2,200; net ₹25,300.

Conclusion

Arithmetic operators and variables automate salary calculations and reduce manual errors.

Validation note

Run the code in Python and compare actual output with the expected result; adjust inputs to test boundary cases.

5. Mobile Plan Cost Optimization

Case study overview and computational thinking

Problem statement

Compare mobile plans against a user's monthly data use and select the lowest-cost eligible plan.

Objectives

- Represent plans as tuples in a list
- Calculate excess-data charges
- Select the lowest eligible cost

Assumptions

- Each plan = (name, price, included_GB, excess_rate_per_GB)
- Usage above included data is charged proportionally
- All plans have 30-day validity

Algorithm

1. Read monthly usage
2. Loop through plans
3. Calculate excess and total cost
4. Choose minimum-cost plan

Computational thinking: break the problem into inputs, rules, calculations, and a clear output.

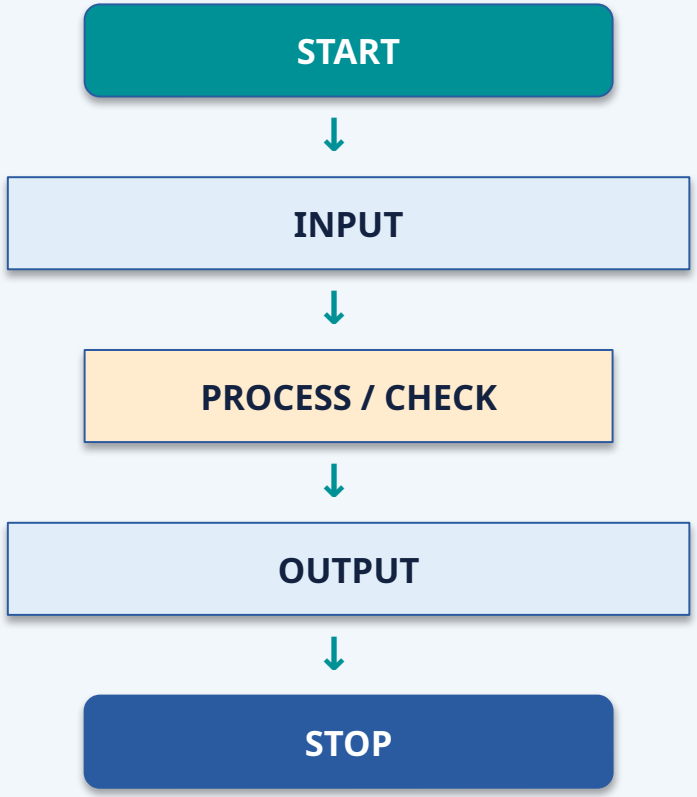
5. Mobile Plan Cost Optimization — Program Design

Pseudocode and flowchart logic

Pseudocode

START → Read usage → For each plan calculate total → Compare totals → Print cheapest plan → STOP

Flowchart (logic)



Decision branches are implemented using if / elif / else.

5. Mobile Plan Cost Optimization — Python Solution

Complete beginner-friendly source code

```
                                                plans = [
    ("Basic", 199, 1.5, 80),
    ("Smart", 299, 3.0, 60),
    ("Max", 399, 6.0, 40)
]
usage = float(input("Monthly data usage (GB): "))
if usage < 0:
    print("Invalid usage")
else:
    costs = []
    for name, price, data, rate in plans:
        extra = max(0, usage - data)
        total = price + extra * rate
        costs.append((total, name))
        print(name, "plan total: ₹", round(total, 2))
    costs.sort()
    print("Most economical plan:", costs[0][1],
          "₹", round(costs[0][0], 2))
```

5. Mobile Plan Cost Optimization — Testing & Conclusion

Example test case and expected result

Test case

Usage 2 GB → Basic ₹239; Smart ₹299; Max ₹399 → Basic is cheapest.

Conclusion

Lists, tuples, loops, and comparisons help users compare plans based on actual usage.

Validation note

Run the code in Python and compare actual output with the expected result; adjust inputs to test boundary cases.

6. Railway Fare and Reservation Calculator

Case study overview and computational thinking

Problem statement

Calculate passenger fares using distance slabs, class multiplier, age concession, and reservation fee.

Objectives

- Use conditional distance slabs
- Apply class and age rules
- Calculate fare for each passenger

Assumptions

- First 100 km: ₹1/km; additional km: ₹0.75/km
- Class multiplier: sleeper 1.0, AC 2.0
- Age 60+ receives 20% concession
- Reservation fee ₹30 per passenger

Algorithm

1. Read distance, class, age
2. Calculate base fare
3. Apply class multiplier
4. Apply concession and add fee

Computational thinking: break the problem into inputs, rules, calculations, and a clear output.

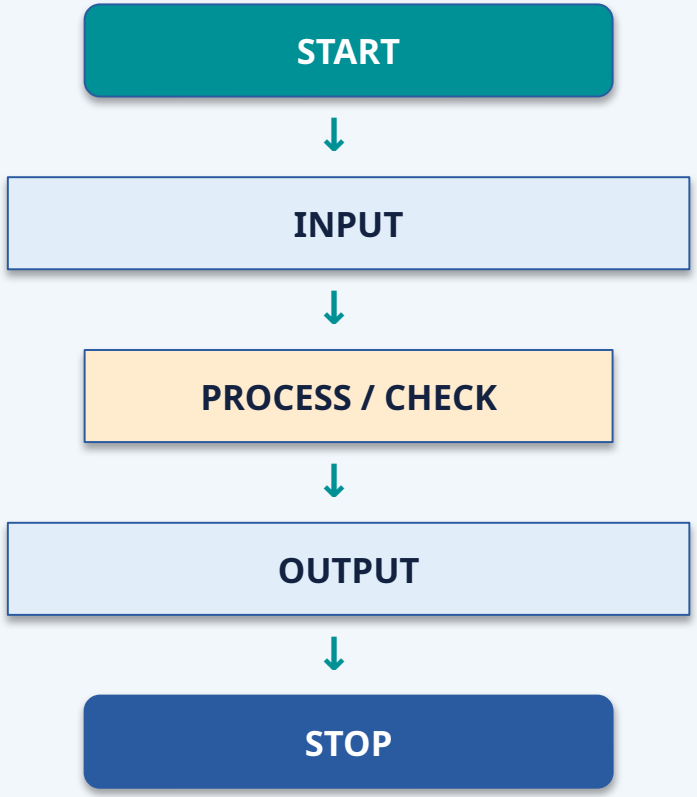
6. Railway Fare and Reservation Calculator — Program Design

Pseudocode and flowchart logic

Pseudocode

START → Read passenger details → Distance fare → Class multiplier → Senior concession? → Add fee → STOP

Flowchart (logic)



Decision branches are implemented using if / elif / else.

6. Railway Fare and Reservation Calculator — Python Solution

Complete beginner-friendly source code

```
                                distance = float(input("Distance in km: "))
age = int(input("Passenger age: "))
travel_class = input("Class (sleeper/ac): ").lower()

if distance <= 0 or age < 0 or travel_class not in ("sleeper", "ac"):
    print("Invalid input")
else:
    if distance <= 100:
        base = distance
    else:
        base = 100 + (distance - 100) * 0.75
    multiplier = 1 if travel_class == "sleeper" else 2
    fare = base * multiplier
    if age >= 60:
        fare *= 0.80
    total = fare + 30
    print("Total fare: ₹", round(total, 2))
```

6. Railway Fare and Reservation Calculator — Testing & Conclusion

Example test case and expected result

Test case

Distance 150 km, age 65, AC → base ₹137.50 × 2 = ₹275; after concession ₹220 + ₹30 = ₹250.

Conclusion

Conditions and arithmetic rules produce a fare based on distance, class, and concession policy.

Validation note

Run the code in Python and compare actual output with the expected result; adjust inputs to test boundary cases.

7. Retail Billing with Discounts and Tax

Case study overview and computational thinking

Problem statement

Create an invoice for several products with quantity, discount, and category-based tax.

Objectives

- Store products in a list
- Loop through items and calculate subtotals
- Apply discount and tax

Assumptions

- Each item has name, price, quantity, tax rate
- Discount 10% when subtotal ≥ ₹1,000
- Discount capped at ₹200
- Tax is calculated after discount

Algorithm

1. Read item details
2. Calculate subtotal
3. Apply threshold discount
4. Calculate tax on discounted amount
5. Print invoice

Computational thinking: break the problem into inputs, rules, calculations, and a clear output.

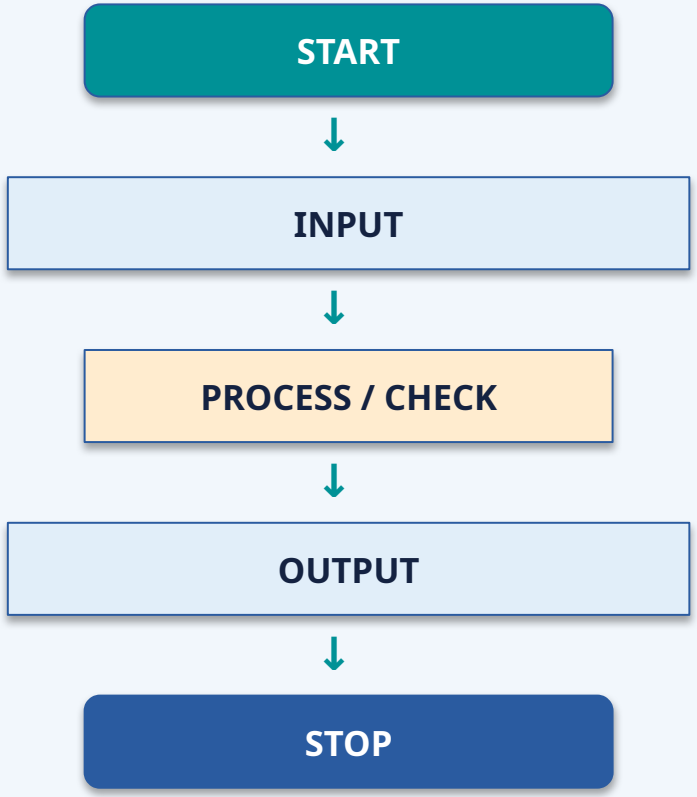
7. Retail Billing with Discounts and Tax — Program Design

Pseudocode and flowchart logic

Pseudocode

START → For each item calculate price × quantity → Sum → Discount → Tax → Final amount → STOP

Flowchart (logic)



Decision branches are implemented using if / elif / else.

7. Retail Billing with Discounts and Tax — Python Solution

Complete beginner-friendly source code

```
                                                items = [
    ("Notebook", 80, 5, 5),
    ("Bag", 700, 1, 12),
    ("Pen", 20, 5, 5)
]
subtotal = 0
for name, price, qty, tax_rate in items:
    item_total = price * qty
    subtotal += item_total
    print(name, ":", qty, "x ₹", price, "=", item_total)

discount = min(subtotal * 0.10, 200) if subtotal >= 1000 else 0
taxable = subtotal - discount
tax = 0
for name, price, qty, tax_rate in items:
    tax += (price * qty) * (tax_rate / 100)
final_amount = taxable + tax
print("Subtotal: ₹", round(subtotal, 2))
print("Discount: ₹", round(discount, 2))
print("Tax: ₹", round(tax, 2))
print("Final invoice: ₹", round(final_amount, 2))
```

7. Retail Billing with Discounts and Tax — Testing & Conclusion

Example test case and expected result

Test case

Subtotal ₹1,200 → discount ₹120; tax based on item rates ₹116; final invoice ₹1,196.

Conclusion

Lists, loops, and arithmetic make itemized billing easier and more accurate.

Validation note

Run the code in Python and compare actual output with the expected result; adjust inputs to test boundary cases.

8. Manufacturing Cost and Profitability

Case study overview and computational thinking

Problem statement

Calculate manufacturing cost including material, labour, machine cost, and wastage; check the profit target.

Objectives

- Calculate total production cost
- Compute unit cost and profit
- Check a target percentage

Assumptions

- Wastage is a percentage of material cost
- Profit target is 20% of total cost
- Selling price is supplied per unit

Algorithm

1. Read quantities and costs
2. Calculate wastage and total cost
3. Calculate revenue and profit
4. Compare profit percentage with target

Computational thinking: break the problem into inputs, rules, calculations, and a clear output.

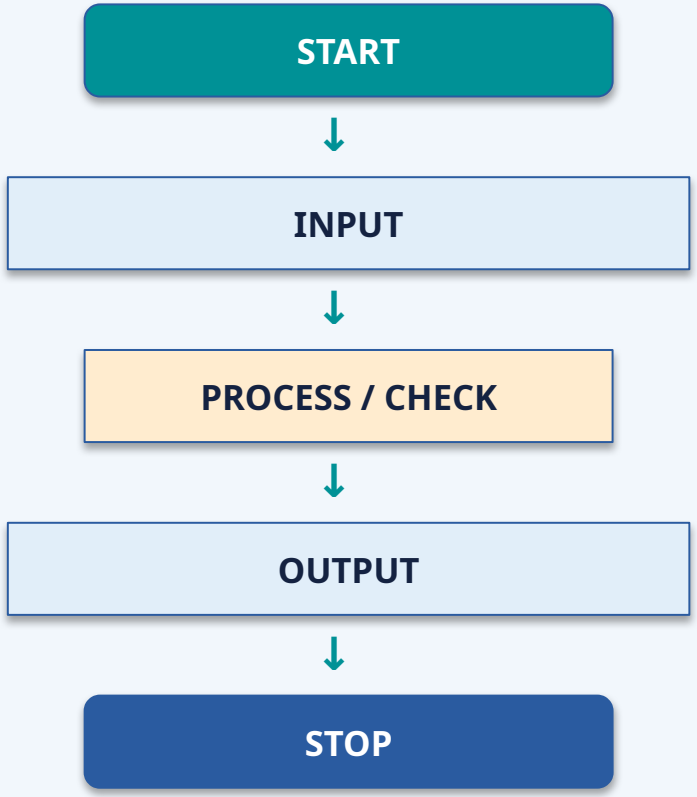
8. Manufacturing Cost and Profitability — Program Design

Pseudocode and flowchart logic

Pseudocode

START → Read inputs → Material + wastage + labour + machine → Revenue → Profit % → Target met? → STOP

Flowchart (logic)



Decision branches are implemented using if / elif / else.

8. Manufacturing Cost and Profitability — Python Solution

Complete beginner-friendly source code

```
                                units = int(input("Units produced: "))
material = float(input("Material cost: ₹"))
labour = float(input("Labour cost: ₹"))
machine = float(input("Machine cost: ₹"))
wastage_rate = float(input("Wastage percentage: "))
selling_price = float(input("Selling price per unit: ₹"))

if units <= 0 or min(material, labour, machine, wastage_rate, selling_price) < 0:
    print("Invalid input")
else:
    wastage = material * wastage_rate / 100
    total_cost = material + wastage + labour + machine
    revenue = units * selling_price
    profit = revenue - total_cost
    profit_pct = (profit / total_cost * 100) if total_cost else 0
    print("Total cost: ₹", round(total_cost, 2))
    print("Profit: ₹", round(profit, 2))
    print("Profit percentage:", round(profit_pct, 2), "%")
    print("Target met:", profit_pct >= 20)
```

8. Manufacturing Cost and Profitability — Testing & Conclusion

Example test case and expected result

Test case

Units 100; material ₹5,000; labour ₹2,000; machine ₹1,000; wastage 10%; selling price ₹100 → cost ₹8,500; revenue ₹10,000; profit ₹1,500 (17.65%); target not met.

Conclusion

Arithmetic and conditional logic show whether a production plan meets its profit target.

Validation note

Run the code in Python and compare actual output with the expected result; adjust inputs to test boundary cases.

9. Student Marks Matrix Analysis

Case study overview and computational thinking

Problem statement

Analyze marks stored in a nested list to calculate student totals, averages, and top performer.

Objectives

- Represent marks as a nested list
- Calculate row-wise totals and averages
- Identify the top-performing student

Assumptions

- Each row stores marks for three subjects
- All marks are out of 100
- Student names correspond to rows

Algorithm

1. Store names and marks
2. Loop through each marks row
3. Calculate total and average
4. Track highest total

Computational thinking: break the problem into inputs, rules, calculations, and a clear output.

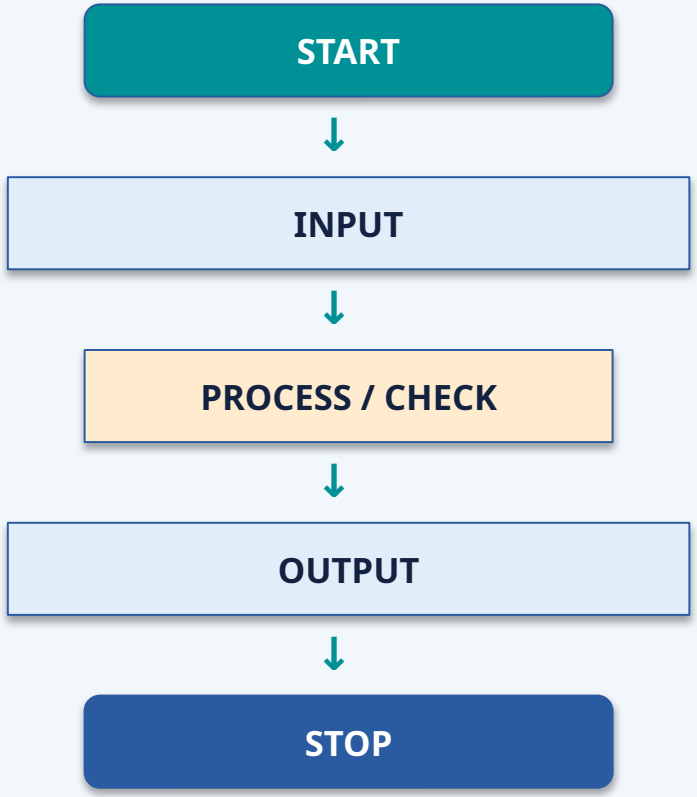
9. Student Marks Matrix Analysis — Program Design

Pseudocode and flowchart logic

Pseudocode

START → Read marks matrix → For each student calculate total/average → Compare totals → Print topper → STOP

Flowchart (logic)



Decision branches are implemented using if / elif / else.

9. Student Marks Matrix Analysis — Python Solution

Complete beginner-friendly source code

```
                                    names = ["Asha", "Ravi", "Meena"]
marks = [
    [78, 85, 90],
    [88, 76, 80],
    [92, 95, 89]
]
top_name = ""
top_total = -1
for i in range(len(marks)):
    total = sum(marks[i])
    average = total / len(marks[i])
    print(names[i], "Total:", total, "Average:", round(average, 2))
    if total > top_total:
        top_total = total
        top_name = names[i]
print("Top performer:", top_name, "with", top_total, "marks")
```

9. Student Marks Matrix Analysis — Testing & Conclusion

Example test case and expected result

Test case

Asha total 253; Ravi total 244; Meena total 276 → Meena is the top performer.

Conclusion

Nested lists and loops organize marks and reveal performance summaries efficiently.

Validation note

Run the code in Python and compare actual output with the expected result; adjust inputs to test boundary cases.

10. Tuple-Based Product Catalogue Audit

Case study overview and computational thinking

Problem statement

Store product details in tuples, validate prices and stock, and identify products needing replenishment.

Objectives

- Use tuples for product records
- Unpack tuple fields
- Calculate inventory value and flag low stock

Assumptions

- Record format = (product ID, name, price, stock)
- Price must be positive and stock non-negative
- Reorder when stock is below 5

Algorithm

1. Loop through product tuples
2. Unpack fields
3. Validate values
4. Calculate stock value and check reorder condition

Computational thinking: break the problem into inputs, rules, calculations, and a clear output.

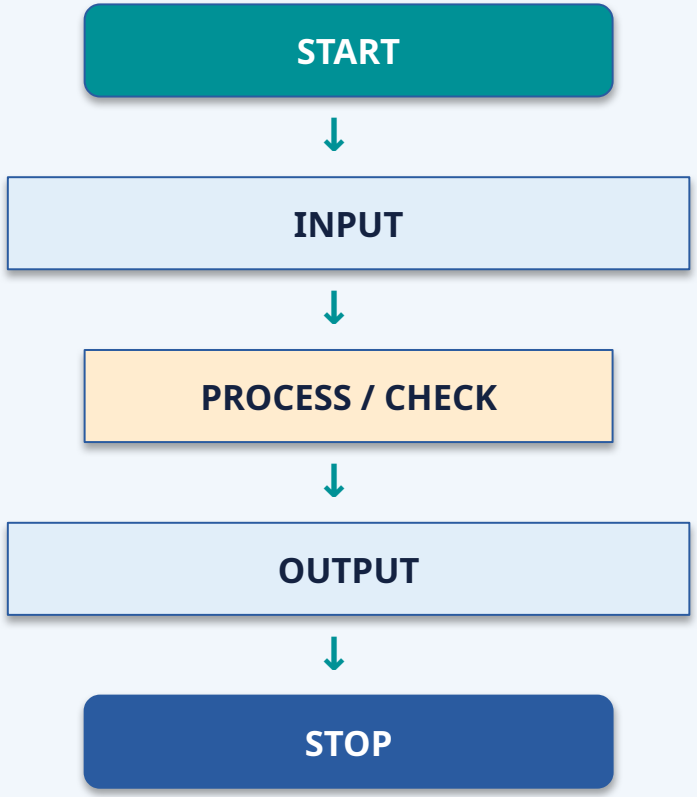
10. Tuple-Based Product Catalogue Audit — Program Design

Pseudocode and flowchart logic

Pseudocode

START → Read product tuples → Unpack each record → Validate → Inventory value → Stock < 5? → Reorder list → STOP

Flowchart (logic)



Decision branches are implemented using if / elif / else.

10. Tuple-Based Product Catalogue Audit — Python Solution

Complete beginner-friendly source code

```
products = [
    ("P101", "Mouse", 450, 12),
    ("P102", "Keyboard", 800, 3),
    ("P103", "USB", 300, 0)
]
reorder = []
inventory_value = 0
for product_id, name, price, stock in products:
    if price <= 0 or stock < 0:
        print("Invalid record:", product_id)
    else:
        value = price * stock
        inventory_value += value
        print(product_id, name, "Value: ₹", value)
        if stock < 5:
            reorder.append(name)
print("Total inventory value: ₹", inventory_value)
print("Reorder products:", reorder)
```

10. Tuple-Based Product Catalogue Audit — Testing & Conclusion

Example test case and expected result

Test case

Mouse value ₹5,400; Keyboard ₹2,400; USB ₹0; total ₹7,800; reorder Keyboard and USB.

Conclusion

Tuples and unpacking keep product records organized and make stock checks straightforward.

Validation note

Run the code in Python and compare actual output with the expected result; adjust inputs to test boundary cases.

THANK YOU

Python Activity-Based Learning

J. Charanya

Roll No: 26000087 | Section: CSE-2

Think logically • Write clearly • Test carefully